

Assignment 1

Tanishq Ganesh Tote 612303190

Que 1: Compare the C and C++ programming languages. Discuss their key differences in terms of syntax, features (structure in C and OOP in C++), and use cases. Provide examples to illustrate your points.

1. Syntax:

- **C:**
 - **Function Declaration:** `int add(int a, int b);`

- **Function Definition:**

```
int add(int a, int b) {  
    return a + b;  
}
```

- **Structure:**

```
struct Point {  
    int x;  
    int y;  
};
```

- **C++:**
 - **Function Declaration:** Same as C.
 - **Function Definition:** Same as C.
 - **Class Definition:**

```
class Point {  
private:  
    int x;  
    int y;  
public:  
    Point(int x, int y) : x(x), y(y) {}  
    int getX() { return x; }  
    int getY() { return y; }  
};
```

2. Features:

- **C:**
 - Procedural Programming
 - Structures to group related variables
 - No support for classes or objects
- **C++:**
 - Object-Oriented Programming (OOP)
 - Classes and Objects
 - Inheritance, Polymorphism, Encapsulation
 - Standard Template Library (STL) for data structures like vectors, maps

3. Use Cases:

- **C:**
 - System programming
 - Embedded systems
 - Low-level applications
- **C++:**
 - Game development
 - GUI applications
 - Real-time systems
 - High-performance applications

Example:

- **C:**

```
#include <stdio.h>

void printHello() {
    printf("Hello, World!\n");
}

int main() {
    printHello();
    return 0;
}
```

- **C++:**

```
#include <iostream>

class Greeter {
public:
    void greet() {
        std::cout << "Hello, World!" << std::endl;
    }
};

int main() {
    Greeter g;
    g.greet();
    return 0;
}
```

Que 2: Study different programming language domains like COBOL, FORTRAN, Lisp, and Prolog. Provide the code snippet along with its explanation, realtime working examples and domain specific comparative studies with other languages.

COBOL code snippets

1. Hello World Program

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. HelloWorld.
```

```
PROCEDURE DIVISION.  
    DISPLAY 'Hello, World!'.  
    STOP RUN.
```

2. Simple Addition

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. SimpleAddition.
```

```
DATA DIVISION.  
WORKING-STORAGE SECTION.  
    77 A PIC 9(2) VALUE 5.  
    77 B PIC 9(2) VALUE 10.  
    77 RESULT PIC 9(2).
```

```
PROCEDURE DIVISION.  
    ADD A TO B GIVING RESULT.  
    DISPLAY 'Result of Addition: ' RESULT.  
    STOP RUN.
```

3. Conditional Statements

```
IDENTIFICATION DIVISION.  
PROGRAM-ID. ConditionalExample.
```

```
DATA DIVISION.  
WORKING-STORAGE SECTION.  
    77 AGE PIC 9(2) VALUE 20.
```

```
PROCEDURE DIVISION.  
    IF AGE >= 18 THEN  
        DISPLAY 'You are an adult.'  
    ELSE  
        DISPLAY 'You are not an adult.'  
    END-IF.  
    STOP RUN.
```

4. Loop Example

IDENTIFICATION DIVISION.
PROGRAM-ID. LoopExample.

DATA DIVISION.
WORKING-STORAGE SECTION.
77 I PIC 9(2) VALUE 1.

PROCEDURE DIVISION.
PERFORM UNTIL I > 5
 DISPLAY 'Iteration: ' I
 ADD 1 TO I
END-PERFORM.
STOP RUN.

5. File Handling Example

IDENTIFICATION DIVISION.
PROGRAM-ID. FileHandlingExample.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
 SELECT INPUT-FILE ASSIGN TO 'input.txt'
 ORGANIZATION IS LINE SEQUENTIAL.

DATA DIVISION.
FILE SECTION.
FD INPUT-FILE.
01 INPUT-RECORD PIC X(100).

WORKING-STORAGE SECTION.
77 EOF-SWITCH PIC X VALUE 'N'.

PROCEDURE DIVISION.
 OPEN INPUT INPUT-FILE.
 PERFORM UNTIL EOF-SWITCH = 'Y'
 READ INPUT-FILE INTO INPUT-RECORD
 AT END MOVE 'Y' TO EOF-SWITCH
 NOT AT END
 DISPLAY INPUT-RECORD
 END-READ
END-PERFORM.
CLOSE INPUT-FILE.
STOP RUN.

PROLOG code snippets

1. Hello World

```
% Hello World in Prolog
hello_world :- write('Hello, World!'), nl.
```

2. Simple Addition

```
% Simple Addition in Prolog
add(X, Y, Result) :- Result is X + Y.
```

3. Conditional Statements

```
% Checking if someone is an adult
is_adult(Age) :-
    Age >= 18 -> write('You are an adult. ');
    write('You are not an adult.').
```

4. List Processing with Recursion

```
% Calculating the length of a list
list_length([], 0).
list_length(_ | Tail, Length) :-
    list_length(Tail, N),
    Length is N + 1.
```

5. File Handling

```
% Reading a file and displaying its contents
read_file(File) :-
    open(File, read, Stream),
    read_file_stream(Stream),
    close(Stream).
```

```
read_file_stream(Stream) :-
    at_end_of_stream(Stream).
```

```
read_file_stream(Stream) :-
    \+ at_end_of_stream(Stream),
    read(Stream, Line),
    write(Line), nl,
    read_file_stream(Stream).
```

6. Fact and Rule Definition

```
% Simple facts about family relationships
parent(john, mary).
parent(mary, susan).
```

```
% Rule to find a grandparent
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

LISP Code snippets

1. Hello World

```
;; Hello World in Lisp
(defun hello-world ()
  (format t "Hello, World!~%"))
```

2. Simple Addition

```
;; Simple Addition in Lisp
(defun add (x y)
  (+ x y))
```

3. Conditional Statements

```
;; Checking if someone is an adult
(defun is-adult (age)
  (if (>= age 18)
      (format t "You are an adult.~%")
      (format t "You are not an adult.~%"))))
```

4. List Processing with Recursion

```
;; Calculating the length of a list
(defun list-length (lst)
  (if (null lst)
      0
      (+ 1 (list-length (cdr lst)))))
```

5. File Handling

```
;; Reading a file and displaying its contents
(defun read-file (filename)
  (with-open-file (stream filename)
    (loop for line = (read-line stream nil)
          while line do
            (format t "~a~%" line))))
```

6. Fact and Rule Definition

```
;; Simple facts about family relationships
(defparameter *parents* '((john . mary)
                           (mary . susan)))

;; Function to find a grandparent
(defun grandparent (x y)
  (some (lambda (z) (and (assoc z *parents*) (equal (cdr (assoc z *parents*)) y)))
        (mapcar #'car (remove-if-not (lambda (p) (equal (cdr p) y)) *parents*)))))
```

FORTRAN code snippets

1. Hello World

```
! Hello World in Fortran
program hello_world
  print *, 'Hello, World!'
end program hello_world
```

2. Simple Addition

```
! Simple Addition in Fortran
program simple_addition
  implicit none
  integer :: x, y, result

  x = 5
  y = 10
  result = x + y

  print *, 'The sum of', x, 'and', y, 'is', result
end program simple_addition
```

3. Conditional Statements

```
! Checking if someone is an adult in Fortran
program check_adult
  implicit none
  integer :: age

  age = 20

  if (age >= 18) then
    print *, 'You are an adult.'
  else
    print *, 'You are not an adult.'
  end if
end program check_adult
```


4. List Processing with Recursion

! Calculating the length of an array (a form of list)

```
program array_length
```

```
  implicit none
```

```
  integer, parameter :: n = 5
```

```
  integer :: array(n), length
```

```
  array = (/1, 2, 3, 4, 5/)
```

```
  length = array_length_recursive(array, n)
```

```
  print *, 'The length of the array is', length
```

```
end program array_length
```

```
integer function array_length_recursive(arr, size)
```

```
  integer, dimension(size) :: arr
```

```
  integer :: size
```

```
  if (size <= 0) then
```

```
    array_length_recursive = 0
```

```
  else
```

```
    array_length_recursive = 1 + array_length_recursive(arr, size - 1)
```

```
  end if
```

```
end function array_length_recursive
```

5. File Handling

! Reading a file and displaying its contents in Fortran

```
program read_file
```

```
  implicit none
```

```
  integer :: i
```

```
  character(len=100) :: line
```

```
  character(len=*) :: filename
```

```
  filename = 'input.txt'
```

```
  open(unit=10, file=filename, status='old')
```

```
  do
```

```
    read(10, '(A)', iostat=i) line
```

```
    if (i /= 0) exit
```

```
    print *, trim(line)
```

```
  end do
```

```
  close(10)
```

```
end program read_file
```

Que 3: What are the functions of registers in assembly language? Study the Accumulator (ACC), Program Counter (PC), Instruction Register (IR), Memory Address Register (MAR), and Memory Data Register (MDR). Provide examples of their applications in assembly code.

1. Accumulator (ACC):

- **Function:** Used for arithmetic and logic operations.
- **Example:**

MOV AX, 5 Load 5 into ACC (AX register)

ADD AX, 3 Add 3 to ACC

2. Program Counter (PC):

- **Function:** Holds the address of the next instruction to execute.
- **Example:**

JMP 0x0040 Jump to address 0x0040

3. Instruction Register (IR):

- **Function:** Holds the current instruction being executed.
- **Example:**

MOV AL, [BX] Load value at address in BX into AL

4. Memory Address Register (MAR):

- **Function:** Holds the address of the memory location to be accessed.
- **Example:**

MOV MAR, 0x0020 Set MAR to address 0x0020

5. Memory Data Register (MDR):

- **Function:** Holds the data read from or written to memory.
- **Example:**

MOV MDR, AL Move data from AL to MDR

Que 4: Comparative study of different architectures like Von Neumann, Harvard, Instruction Set, etc. Come up with real-time examples. Study about the memory hierarchy - Register memory, Cache memory, Main memory along with real-time examples and usage.

1. Von Neumann Architecture:

- **Characteristics:** Single memory space for instructions and data.
- **Example:** Most general-purpose computers.

2. Harvard Architecture:

- **Characteristics:** Separate memory spaces for instructions and data.
- **Example:** Digital signal processors (DSPs).

3. Instruction Set Architecture (ISA):

- **Characteristics:** Defines the set of instructions that a CPU can execute.
- **Example:** x86, ARM.

4. Microarchitecture:

- **Characteristics:** Implementation of ISA in hardware.
- **Example:** Intel Core i7, AMD Ryzen.

Memory Hierarchy:

- **Register Memory:** Fastest, located inside the CPU.
 - **Example:** CPU registers.
- **Cache Memory:** Fast but slower than registers, stores frequently accessed data.
 - **Example:** L1, L2, L3 caches.
- **Main Memory:** Largest and slowest, where programs and data are stored.
 - **Example:** RAM.

Real-time Examples:

- **Von Neumann:** Personal computers.
- **Harvard:** Embedded systems, DSPs.
- **Cache Memory:** Enhances performance by reducing access times.

These studies cover fundamental aspects and applications in each area, illustrating the versatility and differences among programming languages, assembly registers, and computer architectures.